```c
#include "kernel/types.h"   // uint, etc.
#include "kernel/stat.h"    // struct stat (required by user.h)
#include "user/user.h"      // fork, wait, send, recv, printf, exit, getpid

int
main(void)
{
  /* Buffer large enough for one MSG_SIZE (128-byte) message */
  char buf[128];

  /* -------------------------------------------------------
   * Fork a child process.
   * fork() returns 0 to the child, and the child's PID to
   * the parent (following the standard Unix convention).
   * ------------------------------------------------------- */
  int child_pid = fork();

  if(child_pid < 0){
    /* fork() failed — no free process slots */
    printf("msgtest: fork failed\n");
    exit(1);
  }

  /* ==================== CHILD PROCESS ==================== */
  if(child_pid == 0){
    /*
     * We are the child. Our own PID was assigned by the kernel.
     * We call recv() twice to dequeue the two messages the parent
     * is going to send. recv() will BLOCK here until a message
     * arrives if the queue is empty.
     */

    // Custom delay since sleep() is not available in user-space for this branch:
    // Wait for 10 ticks using uptime() so the parent finishes its printfs first.
    int start_time = uptime();
```

```c
    int start_time = uptime();
    while(uptime() - start_time < 10){
      // Busy wait
    }

    /* Receive first message from the queue */
    recv(buf);
    printf("[child]  Received msg 1: %s\n", buf);

    /* Receive second message from the queue */
    recv(buf);
    printf("[child]  Received msg 2: %s\n", buf);

    /* All done — exit cleanly */
    exit(0);
  }

  printf("[parent] My PID is %d. Forked child with PID %d.\n",
         getpid(), child_pid);


  if(send(child_pid, "Hello from the parent! (message 1)") < 0){
    printf("[parent] ERROR: failed to send message 1\n");
    exit(1);
  }
  printf("[parent] Sent message 1 to child.\n");

  /*   * Send message 2 — demonstrates that the queue holds >1 item.   */
  if(send(child_pid, "This is the second message in the queue.") < 0){
    printf("[parent] ERROR: failed to send message 2\n");
    exit(1);
  }
  printf("[parent] Sent message 2 to child.\n");
  wait(0);
  printf("[parent] Child finished. IPC test complete.\n");
  exit(0);
}
```

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ msgtest
[parent] My PID is 3. Forked child with PID 4.
[parent] Sent message 1 to child.
[parent] Sent message 2 to child.
[child]  Received msg 1: Hello from the parent! (message 1)
[child]  Received msg 2: This is the second message in the queue.
[parent] Child finished. IPC test complete.
```